ATIVIDADE PRÁTICA — ESTRUTURAS DE DADOS EM JAVA

Tema: Vetores, Listas, Conjuntos e Mapas
Linguagem: Java
Dificuldade: Intermediária
Objetivo: Compreender as diferenças entre as estruturas de dados `Array`, `ArrayList`, `HashSet` e `HashMap`, por meio da implementação de um sistema de cadastro e consulta de produtos.

1. Contextualização

Uma loja deseja desenvolver um pequeno sistema para controlar seus produtos. O sistema deverá permitir o cadastro de produtos, o armazenamento de seus códigos, a consulta de informações e a identificação dos diferentes produtos cadastrados.

Durante o desenvolvimento, você deverá utilizar quatro estruturas de dados diferentes:

- **Vetor (`Array`):** para armazenar os nomes de produtos em uma estrutura de tamanho fixo.
- **Lista (`ArrayList`):** para armazenar os produtos cadastrados, permitindo adicionar novos elementos.
- **Conjunto (`HashSet`):** para armazenar os códigos dos produtos sem permitir códigos duplicados.
- **Mapa (`HashMap`):** para relacionar o código de cada produto às suas informações, permitindo consultas rápidas pelo código.

O programa deverá ser executado no terminal, utilizando entradas e saídas de texto.

2. Classe Produto

Implemente uma classe chamada `Produto` com os seguintes atributos:

Atributo	Tipo	Descrição
codigo	int	Código identificador do produto
nome	String	Nome do produto
quantidade	int	Quantidade disponível em estoque
preco	double	Preço unitário do produto

A classe deverá possuir:

1. Um construtor para inicializar os atributos.
2. Métodos getters para consultar os atributos.
3. Um método `toString()` para apresentar as informações do produto.

3. Requisitos de implementação

Parte A — Utilizando vetor (`Array`)

Crie um vetor de `String` com capacidade para armazenar exatamente cinco nomes de produtos.

O programa deverá:

1. Solicitar ao usuário o nome de cinco produtos.
2. Armazenar os nomes no vetor.
3. Exibir todos os nomes armazenados.
4. Permitir que o usuário informe uma posição para consultar o nome armazenado nela.

Regras:

- O vetor deverá possuir tamanho fixo de cinco posições.
- As posições deverão ser acessadas por índices, começando em zero.
- O programa deverá validar se o índice informado está entre 0 e 4.

Questão para reflexão: O que acontece quando tentamos inserir um sexto nome no vetor original?

Parte B — Utilizando lista (`ArrayList`)

Implemente uma lista de objetos da classe `Produto`, utilizando `ArrayList<Produto>`.

O programa deverá:

1. Permitir o cadastro de produtos.
2. Permitir que o usuário cadastre quantos produtos desejar, até escolher a opção de encerrar os cadastros.
3. Exibir todos os produtos cadastrados.
4. Informar quantos produtos existem na lista.
5. Permitir a remoção de um produto a partir de seu código.

Regras:

- A lista deverá aceitar produtos com códigos repetidos, caso o usuário os informe.
- A remoção deverá excluir somente o produto correspondente ao código informado, conforme a estratégia definida pelo aluno.
- O programa deverá informar quando um produto não for encontrado.

Questão para reflexão: Por que um `ArrayList` pode ser mais conveniente que um vetor quando não sabemos antecipadamente quantos produtos serão cadastrados?

Parte C — Utilizando conjunto (`HashSet`)

Crie um `HashSet<Integer>` para armazenar os códigos dos produtos.

O programa deverá:

1. Solicitar ao usuário cinco códigos de produtos.
2. Adicionar cada código ao conjunto.
3. Exibir todos os códigos armazenados.
4. Informar a quantidade de códigos distintos existentes no conjunto.
5. Solicitar um código e verificar se ele já está armazenado.

Regras:

- O conjunto não deverá armazenar códigos duplicados.
- Se o usuário informar o mesmo código mais de uma vez, o programa deverá informar que o código já existe.
- Utilize o retorno do método `add()` para verificar se o código foi inserido.

Questão para reflexão: Se o usuário informar os códigos `10`, `20`, `10`, `30` e `20`, quantos elementos existirão no conjunto?

Parte D — Utilizando mapa (`HashMap`)

Crie um `HashMap<Integer, Produto>` para armazenar os produtos, utilizando o código como chave e o objeto `Produto` como valor.

O programa deverá:

1. Cadastrar pelo menos cinco produtos no mapa.
2. Utilizar o código do produto como chave.
3. Consultar um produto a partir do código informado pelo usuário.
4. Exibir os dados completos do produto encontrado.
5. Verificar se determinado código existe no mapa.
6. Exibir todos os produtos armazenados no mapa.
7. Permitir a alteração da quantidade em estoque de um produto.

Regras:

- Cada chave deverá identificar um único produto.
- Caso seja inserido outro produto com uma chave já existente, o aluno deverá tratar a situação e informar o usuário.
- Utilize `containsKey()` para verificar se uma chave existe.
- Utilize `get()` para recuperar o produto correspondente à chave.

Questão para reflexão: O que acontece quando tentamos inserir dois produtos diferentes utilizando o mesmo código como chave?

4. Menu principal

O programa deverá apresentar o seguinte menu no terminal:

```
========== SISTEMA DE PRODUTOS ==========

1 - Trabalhar com vetor
2 - Trabalhar com lista
3 - Trabalhar com conjunto
4 - Trabalhar com mapa
0 - Encerrar o programa

Escolha uma opção:
```

Cada opção deverá permitir a execução das operações correspondentes à estrutura de dados selecionada.

O programa deverá continuar apresentando o menu até que o usuário escolha a opção `0`.

5. Restrições técnicas

1. O programa deverá ser desenvolvido em Java.
2. A classe `Produto` deverá ser implementada separadamente da classe principal.
3. Deverão ser utilizadas as estruturas `Array`, `ArrayList`, `HashSet` e `HashMap`.
4. Não será permitido substituir uma estrutura pela outra.
5. Os dados deverão ser armazenados em memória, sem necessidade de arquivos ou banco de dados.
6. O programa deverá utilizar entrada e saída pelo terminal.
7. O código deverá possuir nomes de variáveis significativos e organização adequada.
8. O programa deverá tratar entradas inválidas e situações em que um produto ou código não seja encontrado.

6. Questões teóricas

Após concluir a implementação, responda:

1. Qual é a principal diferença entre um vetor e um `ArrayList`?

2. Por que o `HashSet` não armazena elementos duplicados?

3. O `HashSet` garante que os elementos sejam exibidos na ordem em que foram inseridos?

4. Qual é a diferença entre armazenar produtos em um `ArrayList` e armazená-los em um `HashMap`?

5. O que acontece quando adicionamos um elemento a um `HashMap` utilizando uma chave que já existe?

6. Qual estrutura é mais adequada quando precisamos consultar frequentemente um produto pelo seu código? Justifique.

7. É possível armazenar dois objetos `Produto` com o mesmo código em um `ArrayList`? E em um `HashMap` que utiliza o código como chave?

8. Se precisarmos armazenar todos os produtos cadastrados e, simultaneamente, garantir que os códigos sejam únicos, quais estruturas poderíamos utilizar em conjunto?

9. Qual é a diferença entre os métodos `add()`, `contains()` e `get()` nas estruturas estudadas?

10. Por que não devemos presumir que um `HashMap` ou um `HashSet` manterá os elementos na ordem em que foram inseridos?

8. Desafio adicional

Modifique o programa para que os códigos dos produtos sejam únicos em todo o sistema.

Para isso, utilize o `HashSet<Integer>` como mecanismo de controle de códigos duplicados durante o cadastro de produtos.

O sistema deverá impedir que um produto seja cadastrado com um código já utilizado, independentemente da estrutura em que o produto será armazenado.

Objetivo do desafio: compreender como diferentes estruturas de dados podem trabalhar em conjunto para resolver um problema.